

Computational Extension of OEIS Sequence A096242: Four New Terms for Base-9 Deletable Primes

Carlo Corti
Independent researcher
carlo.corti@outlook.com

June 2026

Abstract

OEIS sequence A096242 counts n -digit base-9 deletable primes. The local OEIS snapshot used for this project listed the first ten values,

$$4, 14, 58, 221, 911, 3638, 14687, 61435, 262189, 1140171.$$

We report a deterministic bottom-up computation extending the table through $n = 14$:

$$a(11) = 5056361, \quad a(12) = 22726556, \quad a(13) = 103351642, \quad a(14) = 475048816.$$

The computation uses the deletion relation in reverse: every element of D_n must be obtainable by inserting one base-9 digit into an element of D_{n-1} . This gives the explicit raw insertion-attempt count $9n a(n-1)$, an upper bound on the post-leading-zero candidate count C_n , before further filtering, global deduplication, and primality testing. The method is not heuristic. It is supported by replay of the known OEIS values, segmented sort/merge generation for large levels, SHA-256 manifests for the saved levels, and a separate saved-level validity check through $n = 14$. The latter verifies the saved values generated by the production run, but is not a second exhaustive enumeration of missing candidates. As an additional check, the first new level D_{11} was reproduced both by cross-mode production reruns and by a separate GMP-based re-enumerator, yielding a byte-identical level file. No corresponding independent re-enumeration is reported here for D_{12} , D_{13} , or D_{14} .

2020 Mathematics Subject Classification. Primary 11Y55; Secondary 11A41, 11B83.

Key words and phrases. integer sequence, OEIS, deletable prime, base 9, primality testing, computational number theory, segmented enumeration.

1 Introduction

A deletable prime is a prime whose digit string can be reduced by deleting digits, one at a time, while remaining within the same recursively defined class; see the MathWorld overview for the general notion [2]. OEIS sequence A096242 is the base-9 counting sequence for this property:

$$a(n) = \#\{n\text{-digit base-9 deletable primes}\}.$$

The local OEIS snapshot used as source material gives the title “Number of n -digit base-9 deletable primes”, offset 1, 1, and the first ten terms [1]. It records Michael Kleber as the original author, dated February 28, 2003; five further terms from Ryan Propper, dated July 19, 2005; and

a Mathematica program by Robert Price, dated November 13, 2018. The local snapshot was used as the source record for the first ten terms. No separate retrieval date for the local snapshot is recorded in the project artifacts consulted for this manuscript. The human author checked the live OEIS entry during manuscript preparation on June 6, 2026, and found no change relevant to the first ten source terms; no automated live OEIS check was performed by the AI system for this draft.

The OEIS entry places A096242 in the broader deletable-prime family, with cross-references including A080603, A080608, and the base-specific sequences A096235–A096246. This note focuses only on the base-9 member of that family.

The OEIS Mathematica program follows a top-down perspective: for each n , it scans the interval $[9^{n-1}, 9^n)$, selects primes, and checks whether deleting one digit reaches a previously stored deletable prime. The computation reported here uses the same definition but reverses the search. Instead of scanning the full interval, it generates candidates only from D_{n-1} . The resulting candidate count after leading-zero exclusion and before the base-9 last-digit filter and deduplication, denoted C_n below, satisfies

$$C_n \leq 9n a(n-1).$$

The main result is the following finite computational extension.

Computational Result 1 (Validated production computation). *Subject to the documented execution of the production C17 implementation and the recorded validation artifacts described below, the next four values of A096242 after the local OEIS snapshot are*

$$a(11) = 5056361, \quad a(12) = 22726556, \quad a(13) = 103351642, \quad a(14) = 475048816.$$

Corollary 1. *Combining Computational Result 1 with the ten terms in the local OEIS source snapshot gives the proposed b-file extension through $n = 14$ shown in Table 3.*

The rest of the paper explains the definitions, the exact bottom-up enumeration, the base-9 filtering, the segmented implementation, and the validation evidence. Section 2 fixes notation. Section 3 proves the bottom-up generation lemma. Section 4 records the base-9 last-digit filter and the role of global deduplication. Section 5 records sequence-specific features of the base-9 search. Sections 6 and 7 describe implementation and validation. Section 8 gives the resulting table and the proof of Computational Result 1.

2 Base-9 deletable primes

Let $b = 9$, and let $[w]_9$ denote the integer represented by a base-9 digit string w . A deletion is valid only when the resulting string has no leading zero. The recursion is started explicitly from $D_1 = \{2, 3, 5, 7\}$, which is the finite-set version of the OEIS convention that a one-digit prime may terminate the deletion process.

Equivalently, an insertion into an $(n-1)$ -digit string is valid for level n only if the resulting first digit is nonzero; a deletion from an n -digit string is valid only if the resulting $(n-1)$ -digit string is nonempty and has nonzero first digit.

Definition 1. Let

$$D_1 = \{2, 3, 5, 7\}.$$

For $n \geq 2$, an n -digit base-9 integer $p = [w]_9$ belongs to D_n if p is prime and there is a valid deletion of one base-9 digit from w whose resulting $(n-1)$ -digit integer belongs to D_{n-1} . Define

$$a(n) = |D_n|.$$

The deletion relation gives a directed acyclic graph by digit length,

$$D_n \longrightarrow D_{n-1} \longrightarrow \cdots \longrightarrow D_1.$$

This graph is not a tree. A value may have more than one valid deletion parent, and the same candidate may be generated by several insertions. Therefore the enumeration must count distinct values, not insertion paths. Here “parent” means an element of D_{n-1} whose digit string can be reached from the value in D_n by one valid deletion, or equivalently from which the value in D_n can be generated by one valid insertion.

3 Bottom-up generation

Lemma 1 (Bottom-up completeness). *For $n \geq 2$, D_n is exactly the set of primes among the distinct n -digit base-9 integers obtained by inserting one digit $d \in \{0, \dots, 8\}$ into the base-9 representation of an element of D_{n-1} , excluding insertions that create a leading zero.*

Proof. Let $p \in D_n$. By definition, at least one valid deletion of a digit from p gives an element $q \in D_{n-1}$. Re-inserting the deleted digit in the deleted position reconstructs p , and the insertion cannot create a leading zero because p is an n -digit integer. Thus every element of D_n appears in the bottom-up candidate set.

Conversely, suppose an n -digit candidate p is obtained by inserting one digit into some $q \in D_{n-1}$, and suppose p is prime. Deleting the inserted digit gives q , with no leading zero in the resulting string because q is a valid $(n-1)$ -digit base-9 integer. Therefore $p \in D_n$. Passing to distinct generated values removes multiple-generation ambiguity without changing the set. $\square$

The lemma is the mathematical reason the search is exact rather than a probabilistic shortcut. Let C_n denote the number of candidates remaining after the leading-zero exclusion and before the base-9 last-digit filter, deduplication, and primality testing. This C_n is a sizing upper-bound quantity, not the implementation’s post-filter or post-deduplication survivor count. The raw insertion-attempt count before that exclusion is exactly $9n a(n-1)$, so

$$C_n \leq 9n a(n-1),$$

where the factor n counts the possible insertion positions in the resulting n -digit string and the factor 9 counts the inserted digit.

4 Candidate filters and deduplication

Base 9 gives a particularly transparent necessary filter. Since $9 \equiv 0 \pmod{3}$, every base-9 integer is congruent modulo 3 to its last base-9 digit. Therefore a multi-digit prime cannot have final base-9 digit 0, 3, or 6.

Proposition 1 (Last-digit rejection). *For $n \geq 2$, any $p \in D_n$ has final base-9 digit outside $\{0, 3, 6\}$.*

Proof. If the final base-9 digit is 0, 3, or 6, then $p \equiv 0 \pmod{3}$. Such a multi-digit number is greater than 3, so it is composite. $\square$

Thus the only digit-local necessary condition used here is

$$\text{last base-9 digit} \in \{1, 2, 4, 5, 7, 8\}.$$

There is no additional last-digit parity filter in base 9. Since $9 \equiv 1 \pmod{2}$, the parity of a base-9 integer is determined by the sum of its digits modulo 2, not by the last digit alone.

The implementation records the divisibility-by-3 rejection explicitly as the last-digit filter. This is a necessary rejection test, not a primality test. After global deduplication, the implementation applies small-prime trial division by the primes

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97,

and then a deterministic Miller–Rabin strong probable-prime test for the `uint64_t` range. Miller–Rabin is used here in its strong probable-prime form [3, 4]. The implementation uses the seven witnesses

2, 325, 9375, 28178, 450775, 9780504, 1795265022,

which appear in Sinclair’s deterministic-base table and are used here as the implemented seven-witness set for the 64-bit range [5]. The implementation only permits levels up to $n = 20$, so every tested value is below $9^{20} < 2^{64}$. The deterministic sufficiency of this exact seven-witness set for the 64-bit range rests on Sinclair’s computational table, not on a published proof of this particular composite-base set. Peer-reviewed background on strong pseudoprimes and deterministic Miller–Rabin base sets is given by Jaeschke and by Sorenson–Webster [6, 7]; in particular, Sorenson–Webster gives a peer-reviewed deterministic prime-base alternative covering the 2^{64} range. The work of Forišek–Jančina gives additional machine-word primality-testing context and discusses faster alternatives to repeated Miller–Rabin rounds [8].

Global deduplication is indispensable. Since the deletion graph is a DAG, not a tree, the same candidate may be generated from different parents or from different insertion positions. The value $a(n)$ is the cardinality of the set D_n , so repeated insertion paths must not be counted as repeated deletable primes.

5 Sequence-specific features of the base-9 search

The contribution here is a finite exact extension, not a proposed density law for base-9 deletable primes. There is no closed form for $a(n)$ in the project evidence, and no probabilistic model is used to justify the four new terms. The useful specificity is instead structural.

First, the recursive definition gives an exact frontier: to build D_n , one only needs insertions into D_{n-1} . This is a different search space from the top-down interval scan in the OEIS program. Table 1 shows the raw insertion bounds for the new levels and for the next uncomputed level.

Level n	Parent count $a(n-1)$	Bound $9na(n-1)$
11	1,140,171	112,876,929
12	5,056,361	546,086,988
13	22,726,556	2,659,007,052
14	103,351,642	13,022,306,892
15	475,048,816	64,131,590,160

Table 1: Raw bottom-up insertion bounds for the computed extension and the next uncomputed level.

Second, base 9 makes the divisibility-by-3 filter a digit-local condition: the last base-9 digit alone determines the residue modulo 3. This is a small arithmetic feature, but it makes the sequence less generic than a base-independent recursive prime enumeration.

Third, the graph has confluent paths. A candidate that has several valid deletions, or several insertion histories, is still one element of D_n . This is why the reproducible computation is naturally phrased in terms of sort/unique and, at large levels, sorted runs followed by k-way merge.

Finally, the stop at $n = 14$ is a perimeter choice. From the measured value $a(14) = 475048816$, the $n = 15$ insertion bound is

$$C_{15} \leq 9 \cdot 15 \cdot 475048816 = 64131590160.$$

Stored as `uint64_t` values, this is about 513.1 GB in decimal units, or about 477.8 GiB, of raw candidate bytes before filtering, sorting, and merging. A computation of $a(15)$ remains feasible as a larger segmented campaign, but it was outside the scope of the present run.

6 Implementation and segmented search

The production implementation is the C17 source file `src/a096242.c`, built by the accompanying `src/Makefile`. It supports serial and POSIX-threaded execution, in-memory and segmented modes, validation against an OEIS-style b-file, and optional saving of each level as a sorted binary file of little-endian `uint64_t` values. The program rejects `-max-n` values above 20, since $9^{21} - 1$ exceeds the range of the `uint64_t` type.

For smaller levels the program generates the whole candidate level in memory, sorts it, deduplicates it, and then tests primality. For larger levels it uses segmented generation: parent blocks are processed into sorted temporary runs, each run is locally deduplicated, and a global k-way merge removes duplicates spanning different runs while streaming candidates into the primality stage. The merge maintains a min-heap containing the current value from each run; when successive heap outputs have the same value, only the first occurrence is forwarded to the primality stage. The segmented mode changes storage strategy, not the mathematical set being enumerated.

In schematic form, each level is computed as follows:

1. generate insertions from D_{n-1} ;
2. reject leading-zero insertions and final digits 0, 3, and 6;
3. sort and globally deduplicate the surviving candidates;
4. apply small-prime trial division and deterministic Miller–Rabin;
5. write the accepted primes as the sorted level D_n .

In segmented mode, the sort-and-deduplicate step is implemented by sorted temporary runs followed by a k-way merge; duplicates spanning runs are removed during that merge.

The project hardware target recorded in the local dossier was a MinisForum UM790 Pro with an AMD Ryzen 9 7940HS processor, 64 GB DDR5 memory, and 16 hardware threads. This hardware information is included to document the local computation environment, not as a hardware-independent performance claim.

Exact operating-system, kernel, C library, and compiler-version strings for the production run are not present in the local run-evaluation reports consulted for this manuscript; they are therefore not stated here as production metadata.

The release build in the current Makefile uses C17, POSIX threads, strict warning flags, and optimized native compilation:

```
-std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion
-Wdouble-promotion -Wformat=2 -pthread
-O3 -march=native -flto -DNDEBUG
```

The code-review artifacts also record clean GCC and Clang warning-matrix builds. Full wall-clock timings and peak RSS values for the original production run that produced the saved D_{11} through D_{14} levels are not included in the local run-evaluation reports; they are therefore not reported here. The local artifacts consulted here do not record a separate machine identity for the D_{11} validation reruns. The D_{11} timings reported below are validation-rerun measurements, not recovered original-production timings or hardware-independent benchmarks.

7 Validation and reproducibility

Validation was organized around the production source, the extended b-file `data/b096242.txt`, the run-evaluation reports for $n = 13$ and $n = 14$, the manifest `levels_manifest.tsv`, and the separate saved-level certifier `cert/a096242_level_certifier.c`.

Throughout this section, paths of the form `levels/DNN.bin` refer to saved binary level files produced during the local computational campaign. Those multi-GB generated artifacts are not included in the minimal public GitHub package. Their counts, byte sizes, and SHA-256 hashes are documented in `levels_manifest.tsv` and in the validation reports.

The known OEIS values through $n = 10$ were replayed by serial, parallel in-memory, and forced segmented runs, with counts matching the local OEIS snapshot as recorded in the local validation summary. The code-review artifacts record release, debug, and sanitizer builds; regression tests; deterministic comparison of saved levels for different thread counts through D_{10} ; ASan/UBSan, TSan, and Valgrind runs; and static-analysis checks.

After the manuscript review process identified the absence of a high-level cross-mode check, the first new level D_{11} was regenerated by the production program in two deliberately different modes: single-thread in-memory mode and 16-thread segmented mode with parent chunks of size 10000. Both regenerated files were byte-identical to the saved `levels/D11.bin` file. The shared SHA-256 hash was

`38ffc99bf4836f55...af0dd3a59.`

The hash is abbreviated in the display; full 64-character SHA-256 values are recorded in `levels_manifest.tsv`. The single-thread in-memory rerun reported $a(11) = 5056361$, wall time 0:19.48, and peak RSS 1718048 kB. The 16-thread segmented rerun reported $a(11) = 5056361$, wall time 0:10.56, and peak RSS 102848 kB.

The first new level was also re-enumerated by a separate C validation program, `cert/a096242_d11_gmp_reenumerate.c`. This program reads `levels/D10.bin`, enumerates all raw D_{11} insertion attempts, applies the leading-zero and base-9 last-digit filters, globally deduplicates the surviving candidates, applies the same small-prime trial-division pre-filter through 97, and then calls GMP's `mpz_probab_prime_p` with repetition parameter 25 rather than using the production Miller–Rabin routine. The local GMP package version was 6.3.0. The run reported 112,876,929 raw insertion attempts, 87,478,862 globally unique candidates after the leading-zero and base-9 last-digit filters and global deduplication, 14,288,558 GMP primality calls, and 5,056,361 accepted values. The resulting `D11_gmp.bin` file was byte-identical to `levels/D11.bin`, with the same SHA-256 hash above. This is an independent probabilistic primality cross-check, not a deterministic primality certificate. The GMP re-enumeration ran in 0:18.07 wall time with peak RSS 1,625,204 kB.

These D_{11} checks strengthen the evidence for the first new term and for the production pipeline’s memory/segmented equivalence at a new level. They are still not an exhaustive independent re-enumeration of D_{12} , D_{13} , or D_{14} . For those levels, the present validation consists of the production enumeration, manifest hashes, structural file checks, and the saved-level validity certifier described below.

The saved binary files `levels/D01.bin` through `levels/D14.bin`, representing D_1 through D_{14} , were checked for file-size consistency, strict increasing order, absence of duplicates, and membership in the correct base-9 digit-length interval. The manifest records the count, byte size, and SHA-256 hash of each level. Table 2 lists the four new levels.

Level	Count	Bytes	SHA-256 prefix
11	5,056,361	40,450,888	38ffc99bf4836f55...af0dd3a59
12	22,726,556	181,812,448	42c916e29431c05c...207c40a7c
13	103,351,642	826,813,136	69ff460d9198fbe4...e95126595
14	475,048,816	3,800,390,528	13d23872161dcb27...8ba91aa4

Table 2: Manifest entries for the four new levels. The full SHA-256 values are stored in `levels_manifest.tsv`.

A separate saved-level certifier was run through $n = 14$. It loads the saved levels, verifies $D_1 = \{2, 3, 5, 7\}$, checks sorted order and digit-length range, and for every saved value in $D_2, \dots, D_{14}$ checks primality and the existence of at least one valid base-9 digit deletion into the previous saved level. The report records

CERTIFIER RESULT: PASS through n=14.

This certifier supports saved-level validity. It does not perform a second exhaustive enumeration proving that no candidate is missing. Since the definition of a deletable prime requires the existence of at least one valid deletion into the previous level, this parent-existence check is sufficient for membership of a saved value, conditional on primality and on the previous saved level.

The certifier is independent of the production program as a separate C source file and saved-level traversal. Its primality predicate uses the same deterministic 64-bit Miller–Rabin witness set as the production code, so it should not be read as an independent primality-method certificate such as an ECPP proof or a GMP-based oracle. Its independent contribution is the separate pass over the saved files: range and order checks, duplicate rejection, parent-deletion lookup, and per-value validation against the previous saved level.

8 Results

The proposed extended b-file is shown in Table 3. The first ten values are the values in the local OEIS snapshot and were replayed by the implementation. The last four values are the new computed terms.

n	$a(n)$
1	4
2	14
3	58
4	221
5	911
6	3,638
7	14,687
8	61,435
9	262,189
10	1,140,171
11	5,056,361
12	22,726,556
13	103,351,642
14	475,048,816

Table 3: Values of A096242 through $n = 14$.

Proof of Computational Result 1. The mathematical completeness argument is inductive. The base level is $D_1 = \{2, 3, 5, 7\}$. If the saved level D_{n-1} is complete, then Lemma 1 says that D_n is obtained by considering the distinct valid insertions into D_{n-1} and retaining exactly the primes. Proposition 1 gives a necessary last-digit rejection for multi-digit candidates in base 9, and it removes no prime. The production implementation follows this induction level by level, with global sort/merge deduplication before primality testing.

The first ten counts reproduce the local OEIS b-file. The saved levels D_{11} through D_{14} have the counts shown in Table 2, and the full count/byte/hash records are stored in `levels_manifest.tsv`. The separate saved-level certifier verifies every saved value through D_{14} as prime with a valid deletion into the previous level. In addition, D_{11} was reproduced byte-for-byte by single-thread memory and 16-thread segmented production reruns and by a separate GMP-based re-enumerator. These checks strengthen the first new level and the pipeline reproducibility evidence, while completeness of D_{12} through D_{14} remains tied to the documented production enumeration. Subject to the correctness of that enumeration and validation pipeline, the cardinalities of these four levels are therefore the four stated values. $\square$

Proof of Corollary 1. The local OEIS source snapshot lists values through $n = 10$. Computational Result 1 supplies the next four values computed by the present project. Combining these two sources gives exactly the fourteen rows shown in Table 3. $\square$

9 Conclusion

The bottom-up formulation turns the recursive definition of base-9 deletable primes into an exact level-by-level enumeration. Applied to A096242, it extends the known table from $n = 10$ to $n = 14$, adding

$$5056361, \quad 22726556, \quad 103351642, \quad 475048816.$$

The computation is best read as a finite certified table extension, not as a growth law for deletable primes. Its distinctive features are the exact parent-frontier generation, the base-9 last-digit filter,

the need for global deduplication in a level DAG, and the manifest-based reproducibility of large generated levels.

Data availability

The code, extended b-file, validation reports, saved-level certifier, and level manifest are part of the local project package. At this draft stage they are GitHub-ready local artifacts, not yet a public immutable archive. They are to be included in the project repository and archive listed in the References when those public artifacts are finalized. The principal artifacts are the C17 source and Makefile, `data/b096242.txt`, `levels_manifest.tsv`, the run-evaluation reports, the saved-level certifier source, and the GMP-based D_{11} re-enumerator source `cert/a096242_d11_gmp_reenumerate.c`. The binary files `levels/D01.bin` through `levels/D14.bin` are generated local artifacts, not intended contents of the minimal GitHub repository. They are regenerable from the source code and checkable against the manifest counts, byte sizes, and SHA-256 hashes. Before final archival release, the public package should also record any available final commit or tag, Zenodo DOI, and any recovered production environment metadata or original-production timing/RSS records. A local Markdown mirror of Sinclair’s witness table is included in the GitHub-ready package as `references/sinclair_miller_rabin_sprp_bases.md`.

AI use disclosure

This work used large language models as research assistants during the workflow for the A096242 project. Gemini was used in the initial triage of the sequence and project feasibility. An OpenAI Codex system, identified by the working environment as GPT-5, was used during code development, code review, validation planning, manuscript drafting, and manuscript revision. Additional LLM systems identified as Claude, DeepSeek, Grok, Kimi, Qwen, and Z.ai were used as non-author reviewers during code and manuscript review rounds.

The AI-assisted work included mathematical analysis of the base-9 deletable prime definition, formulation of the bottom-up insertion method, recognition of the base-9 last-digit filter, C17 implementation support, review triage, validation planning, interpretation of run reports, and drafting of this manuscript.

Mathematical analysis. The bottom-up completeness lemma, the last-digit filter, and the DAG/deduplication interpretation were developed and written with AI assistance. The initial sequence triage used Gemini; the formulation and manuscript presentation of the bottom-up lemma, DAG interpretation, and validation boundaries were developed mainly in Codex-assisted sessions. The identities and set-theoretic claims used in the paper are proved explicitly in the text and were checked against the final implementation and saved-level certifier before inclusion. The human author independently verified the logical correctness of each proof and the validity of each algorithmic step before accepting the final text.

Algorithm and code. The production C17 search program, Makefile, saved-level certifier, and GMP-based D_{11} re-enumerator were produced through an AI-assisted development and review workflow. Their correctness does not rest on that provenance: it rests on known-term replay, memory/segmented-mode validation, sanitizer and tooling checks, deterministic saved-level comparisons, run-evaluation reports, manifest hashes, the separate saved-level validity check, and the independent GMP-based D_{11} re-enumeration described in this paper.

Manuscript. The manuscript was drafted with AI assistance and revised by the author. AI systems supplied drafts, edits, and review suggestions; the final verifying revision and acceptance of

the text were performed by the human author. Every quantitative claim included here was checked against the local computational artifacts: the OEIS source snapshot, the extended b-file, the level manifest, the run-evaluation reports, the GitHub-ready validation summary, and the certifier report.

The author takes full scientific responsibility for the mathematical statements, the algorithm, the implementation, the numerical results, the validation boundaries, and the conclusions of this paper.

References

- [1] OEIS Foundation Inc., *Sequence A096242: Number of n -digit base-9 deletable primes*, The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A096242>. Sequence created by Michael Kleber, Feb. 28, 2003; five more terms from Ryan Propper, Jul. 19, 2005; Mathematica program by Robert Price, Nov. 13, 2018. Local project snapshot used as source material; live entry checked by the author on June 6, 2026.
- [2] Eric W. Weisstein, *Deletable Prime*, MathWorld—A Wolfram Web Resource. <https://mathworld.wolfram.com/DeletablePrime.html>. Accessed June 6, 2026.
- [3] Gary L. Miller, *Riemann’s hypothesis and tests for primality*, Journal of Computer and System Sciences 13 (1976), 300–317. doi:10.1016/S0022-0000(76)80043-8.
- [4] Michael O. Rabin, *Probabilistic algorithm for testing primality*, Journal of Number Theory 12 (1980), 128–138. doi:10.1016/0022-314X(80)90084-0.
- [5] Jim Sinclair, *Strong pseudoprime bases*, online table of deterministic Miller–Rabin bases. <https://miller-rabin.appspot.com/>. Accessed June 6, 2026. A local Markdown mirror of the cited table is included in the GitHub-ready package as `references/sinclair_miller_rabin_sprp_bases.md`.
- [6] Gerhard Jaeschke, *On strong pseudoprimes to several bases*, Mathematics of Computation 61 (1993), 915–926. doi:10.1090/S0025-5718-1993-1192971-8.
- [7] Jonathan P. Sorenson and Jonathan Webster, *Strong pseudoprimes to twelve prime bases*, Mathematics of Computation 86 (2017), no. 304, 985–1003. doi:10.1090/mcom/3134.
- [8] Michal Forišek and Jakub Jančina, *Fast primality testing for integers that fit into a machine word*, SOFSEM 2015 Student Research Forum Papers and Posters, CEUR Workshop Proceedings 1326 (2015), 20–30. <https://ceur-ws.org/Vol-1326/020-Forisek.pdf>.
- [9] C. Corti, *A096242: Computational package for OEIS sequence A096242*, planned GitHub repository and Zenodo archive, 2026. <https://github.com/carcorti/A096242>. Zenodo DOI to be assigned after final synchronization and archival.